DASE7506 MP2
Experimental Report

Li Qingchen 3036746351

1 Project Objective and Final Results

This project selects one of four fixed candidate solutions for each mathematics question, using the supplied texts and 1024-dimensional embeddings. It generates no new answers and calls no external model. The final method selects a correct answer for 395 of 467 test questions, achieving 84.5824%, compared with 376/467 and 80.5139% for the initial method. This is an improvement of 19 questions and 4.0685 percentage points. Errors fall from 91 to 72, a relative reduction of 20.88%.

Metric	Initial	Final
Validation correct	747/798	758/798
Validation accuracy	93.6090%	94.9875%
Test correct	376/467	395/467
Test accuracy	80.5139%	84.5824%
Local prediction plus scoring	0.528 s	1.471 s
Uncompressed inference assets	557,939 bytes	3,754,806 bytes

Task Definition and Data Boundaries

The training, validation and test sets contain 4,755, 798 and 467 questions. Each question has four candidates; the method returns one Python integer from 0 to 3. Correctness depends on whether the final number extracted by the official parser equals the reference. Several candidates may be correct. Prediction does not read references, and IDs are used only for alignment. A correct final number does not establish that every reasoning step is valid.

The project prohibits additional datasets, pretrained weights, API calls and newly generated candidates. Complete CPU prediction plus scoring must use at most four threads and finish within 30 seconds. Uncompressed inference assets must not exceed 16 MiB. The local measurements meet these numerical limits.

Scope of the Results

Twelve frozen versions, including the initial and final methods, were scored during development. The final score is reproducible, but it is not an unbiased estimate from a single untouched blind test. Model fitting and rule development did not inspect test references question by question. Nevertheless, repeated aggregate feedback and stopping after reaching the target may cause test adaptation. A fresh independent audit bank is needed to assess generalization further.

2 Baselines and Connections to the Lectures

From Simple Rules to the Initial Reward Scorer

Training references provide correctness labels for the four candidates. Comparisons include selecting the first candidate, majority voting, style features, embedding scorers and models using within-question context. All results in the table below are validation results.

Validation method	Correct	Accuracy
Select the first candidate	624/798	78.1955%
Parsed-answer majority vote	688/798	86.2155%
Style-only linear model	675/798	84.5865%
Embedding linear model	726/798	90.9774%
Embedding MLP	732/798	91.7293%
Context pairwise scorer seed 43	747/798	93.6090%
Final OOF combiner	758/798	94.9875%

The initial model is an MLP with 64 hidden units. It maps embeddings, text and answer features, and within-question differences to a scalar score. Its pairwise loss places correct candidates above incorrect ones, following the Bradley–Terry preference model: apply a negative log-sigmoid loss to the difference between their scores. Seed 43 performed best among the compared candidates and was exported as the initial model; the three seeds were not simply averaged.

Why the Initial Method Still Fails

Fluent language, numerous steps and a popular answer do not guarantee correctness. The initial embedding model detects some errors but is less effective at using same-answer relationships, local arithmetic contradictions and minority correct answers. Pairwise ranking also excludes training questions whose four candidates are all correct, losing supervision about what correct solutions look like.

Lecture topic	Use in this project
Lecture 5 Persuasiveness and verification	Parse final answers, check local arithmetic, and analyze semantically wrong but arithmetically valid solutions
Lecture 6 Self consistency and reward models	Voting baseline, numerical answer groups, pairwise ranking and correctness scoring
Lecture 7 Proper scoring and calibration	Select combinations by correct probability mass NLL on hard validation questions, alongside accuracy
Lecture 8 Hard and soft targets	Explore distribution distillation for small scorers; excluded from the final method

This project does not train a generative language model or implement PPO or GRPO. The final method performs supervised candidate selection and combination.

3 Answer Verification and Numerical Evidence

From Persuasiveness to Checkable Evidence

Lecture 5, page 8, emphasizes that an orderly explanation need not be sound. Pages 41–42 and the subsequent verification section describe extracting the final answer, normalizing it and checking equivalence with a reference. This project follows that principle: final-number correctness defines training labels, while checkable evidence in the text supports prediction. The official parser and scorer remain unchanged.

Reference Verification and Reference-Free Scoring

References are available for training and evaluation, but not to the prediction method. At prediction time, correctness must be estimated. Lecture 6, page 27, distinguishes these settings. Consequently, the test-time candidate scorer cannot compare against the gold answer, and a local arithmetic check cannot establish the validity of a complete solution.

Stage	Project implementation
Extraction and normalization	Use the official parser to compare equivalent numerical representations
Supervision	Match parsed candidate values to training references; multiple candidates may be correct
Local numerical evidence	Check parseable arithmetic, number coverage and final-answer statistics
Prediction and evaluation	Select using inputs only; the official scorer subsequently checks final values independently

Turning Verification into Learnable Signals

The implementation extracts 64 auxiliary features covering text statistics, answer–question relationships, loose numerical checks and 12 strict arithmetic checks. It also computes each candidate's difference from the within-question mean. The feature counts and parsing rules are engineering extensions; the lecture supplies the verification principle rather than this exact feature design.

For example, a statement such as "$6 \times 7 = 48$" supplies a clear local error signal. Fractions, algebra and incomplete expressions may instead trigger false alarms, so strict checks consider only reliably parsed numerical fragments. In training, these checks flag anomalies in approximately 1.65% of correct candidates and 8.74% of incorrect candidates. They are informative features, not hard rules for discarding candidates.

Limits of Verification

Correct local arithmetic does not ensure correct interpretation. For example, 70 days must first become 10 weeks before "6 hours per week" yields 60 hours. The incorrect calculation $6 \times 70 = 420$ is arithmetically valid but uses the wrong units. This illustrates Lecture 5's discussion of verification scope and false positives. Scorers therefore also use embeddings and relationships between candidates. To control resources, some branches use the first 128 components of a training-fitted PCA, while others retain the original embeddings; dimensionality reduction is an implementation aid.

4 Self Consistency and Reward Scoring

Self Consistency as a Baseline

Lecture 6, page 18, presents sampling, answer extraction and voting. This project already supplies four candidates, so only extraction and voting are used; no extra solutions are generated. Majority voting achieves 688/798, or 86.2155%, on validation, compared with 78.1955% for selecting the first candidate. Agreement across reasoning paths is therefore useful.

Agreement can also reflect a shared mistake. If candidates answer 8, 4, 8 and 8, majority voting chooses 8. In one validation example, relative speed and distance imply a correct time of 4 minutes. The project therefore learns the credibility of answer groups, allowing a minority answer to win.

Extending Voting to Answer Group Scoring

Models B and E group candidates by their parsed numerical answers, aggregate same-answer representations and whole-question context, and assign a shared score within each group. The values 128 and 64 denote two internal representation widths. Training reduces the mechanical advantage of repeated answers and gives extra weight to questions with only one correct candidate. This extends the self-consistency idea; the particular network and loss are project-specific rather than direct implementations from the slides.

Bradley Terry Pairwise Learning

Lecture 6, page 51, models preference using the score difference between a preferred candidate and another candidate. The initial method constructs correct–incorrect pairs from training references and raises the correct score relative to the incorrect score. Here r denotes the scorer, with plus and minus marking correct and incorrect candidates.

$$P(correct\ candidate\ wins) = \sigma(r_+ - r_-)$$

$$L = -log\ \sigma(r_+ - r_-)$$

The initial scorer uses a small MLP rather than the LLM reward head illustrated in the lecture. Supervision comes from numerical correctness rather than human preferences. It reuses the pairwise mathematical objective without subsequent PPO training. It achieves 747/798, or 93.6090%, on validation and 376/467, or 80.5139%, on test.

Adding Final Outcome Supervision

Lecture 7, pages 6 and 12, discusses verifiable outcomes and final-answer supervision. Model A applies this idea through correctness classification over all candidates. The 2,242 all-correct training questions provide no positive–negative ranking pairs, but supply 8,968 positive examples. A also removes three vote-count features, retaining a view that depends less on the majority answer. This is supervised classification, not RLVR or GRPO.

The final system also retains robust scorer C and neural-plus-tree scorer D, giving five complementary branches. Label smoothing, parameter averaging and trees are engineering enhancements; their roles are summarized here, while the source code and frozen weights retain the implementation details.

5 Proper Scoring Rules and Final Decisions

Why Accuracy Alone Is Insufficient

Lecture 7, page 51, discusses Brier and logarithmic scores: probability quality matters in addition to whether the chosen answer is correct. Two models can both miss a question while allocating 0.40 and 0.01 to the correct answer, expressing very different degrees of erroneous confidence. Looking only at the highest-probability candidate hides this difference. This project uses proper-score ideas for validation selection; it does not implement the lecture's RLCD reinforcement-learning updates.

Aggregating Probability over Correct Candidates

A question may have several correct candidates, so one candidate should not be treated as the unique correct class. If answers are 4, 4, 8 and 8 and the reference is 4, both of the first two probabilities support the correct answer. Let y denote candidate correctness labels, p candidate probabilities, and M the total probability assigned to correct candidates.

$$M_i = \Sigma_j \, y_{ij} \, p_{ij}$$

$$NLL = -\frac{1}{N} \, \Sigma_i \, log(M_i)$$

This adapts the lecture's log-score idea to a selection task with several acceptable solutions. Lower correct probability mass incurs a larger negative-log penalty. The score compares probability assignments, but validation selection does not guarantee perfectly calibrated probabilities.

Selecting Combinations on Hard Validation Questions

When all four candidates are correct, any selection earns a point and can hide differences in selection ability. The main selection criterion therefore uses 355 validation questions with at least two distinct valid parsed answers. Development explored Brier and NLL; the final combination and mixture weight were selected by hard-question NLL, with accuracy as a secondary criterion. With the same five base models, equal averaging gives NLL 0.362074; the final combination reduces this to 0.352660 and increases validation correct answers from 755 to 758.

Probability Aggregation and Final Inference

Each base model first normalizes its score scale within a question, masks candidates with unparseable answers and applies softmax. Lecture 7, pages 58 and 61, discusses temperature scaling. The specific normalization here is an engineering choice, not the same as fitting a temperature, and it does not guarantee calibration. Final probabilities use the following mixture:

$$p_{final} = 0.5 \, p_{meta} + 0.5 \, p_{mean}$$

Here mean is the equal probability average of the five base models, and meta is the small combiner's output. The selected index maximizes the final probability. The combiner has depth 3 and 100 trees and is fitted to four-fold out-of-fold predictions within training, reducing direct reliance on optimistic in-sample predictions. OOF and trees are additional engineering methods. PCA is pre-fitted on all unlabeled training inputs, so this is not strictly nested cross-validation of every preprocessing step.

6 Validation Comparisons and Error Analysis

Comparing Combinations of the Same Base Models

To isolate the combination step, equal averaging and the final 0.5 mixture are compared using exactly the same five base models. The hard subset contains 355 validation questions with at least two distinct valid parsed numerical answers.

Metric	Equal base-model average	Final combination
Validation correct	755/798	758/798
Validation accuracy	94.6115%	94.9875%
Hard validation correct	312/355	315/355
Hard validation accuracy	87.8873%	88.7324%
Hard-set correct-mass NLL	0.362074	0.352660

The final combination gains three correct answers and reduces NLL by approximately 2.60%. NLL penalizes assigning very little probability to correct answers and distinguishes overconfident combinations with similar accuracy. Because the metric also drives selection, this is a development-set comparison rather than an independent significance test.

Correct candidates per question	Questions	Average correct	Final correct
1	117	88	87
2	123	113	114
3	179	175	178
4	379	379	379

The final method does not improve every subgroup: it gets one fewer single-correct-candidate question right than equal averaging. Against the initial method, however, this subgroup improves from 72/117 to 87/117, or 61.54% to 74.36%. Validation includes 379 all-correct questions, so the overall 94.99% accuracy makes difficult selection look easier than it is.

Validation Cases and Remaining Limitations

Success: a relative speed of 24 − 15 = 9 miles per hour and a 0.6-mile head start imply a catch-up time of 4 minutes. The four answers are 8, 4, 8 and 8. The final method selects the minority answer 4, whereas the initial model fails. This shows that learned verification signals can resist an incorrect majority, but one case cannot quantify any individual branch’s contribution.

Failure: a person naps three times per week for two hours each time. Asked for the total over 70 days, the final method selects 420 rather than 60, which requires converting days to 10 weeks. Both $3 \times 2 = 6$ and $6 \times 70 = 420$ are locally correct; the error lies in units and interpretation. Another class-pairing example treats $94 - 46 = 48$ as the remainder, overlooking the other member consumed by each pair. Local checks cannot prove the entire reasoning valid.

7 Deployment and Reproduction

Keeping Only Assets Required for Prediction

PCA parameters, neural weights, tree models and configuration are all part of the final implementation. They transform representations, score candidates and combine predictions.

Item	Final record
Scoring and combination	Five base scorers and one combiner
Loaded files and total size	Nine files; 3,754,806 bytes, about 3.58 MiB
Asset limit	16 MiB; about 22.38% used
Local prediction plus scoring	1.471292 s; four-thread cap
Inference dependencies	NumPy SciPy CatBoost; no PyTorch or GPU

Timing covers startup, imports, input loading, every scoring branch and the independent scorer in two fresh processes. Prediction takes 1.271396 seconds and scoring 0.199893 seconds. The local machine runs Windows and Python 3.12.7 on an AMD Ryzen 7 7800X3D with 8 cores and 16 logical threads.

Reproducing the Score with Frozen Weights

Use the matching frozen weights without retraining. Copy the original course repository's complete code/data directory into this repository's code/data directory; original datasets are not duplicated in the minimal submission. Open PowerShell at the repository root and run:

```
cd code
py -3.12 -m venv .venv-improved
.\.venv-improved\Scripts\python.exe -m pip install `
  -r requirements-improved-inference.txt
$env:OMP_NUM_THREADS='4'
$env:OPENBLAS_NUM_THREADS='4'
$env:MKL_NUM_THREADS='4'
$env:NUMEXPR_NUM_THREADS='4'
.\.venv-improved\Scripts\python.exe check_data.py
.\.venv-improved\Scripts\python.exe verify_stack.py
.\.venv-improved\Scripts\python.exe evaluate_stack.py `
  --split test --output-dir reproduced_stack
```

Expected Outputs and Checks

The file reproduced_stack/test-score.json should contain n=467, correct=395 and accuracy=84.58244111349036. Predictions contain one index from 0 to 3 per question. Runtime varies with hardware, load and caches, so it need not equal 1.471292 seconds. Changing split to validation should yield 758/798 and 94.9874686716792%.

The evaluation helper invokes the official entry points below. To inspect prediction and scoring separately, set the virtual-environment Python path and run:

```
$mp2py='.\.venv-improved\Scripts\python.exe'
& $mp2py predict.py --method submission_stack.py `
  --checkpoint model_bundle_stack --split test `
  --output reproduced_stack/test-predictions.json
& $mp2py score_predictions.py --split test `
  --predictions reproduced_stack/test-predictions.json `
  --output reproduced_stack/test-score.json
```

8 Retraining Scope Evidence and Disclosure

Refitting the Combiner versus Training Everything from Scratch

From a clean copy's code directory, the following process uses the supplied training-fitted PCA and base weights, prepares training and validation features, regenerates four-fold base predictions and refits the second-level combiner. It does not read test data or overwrite the frozen final model directory.

```
$mp2py='.\.venv-improved\Scripts\python.exe'
& $mp2py -m pip install -r requirements-improved-training.txt
& $mp2py prepare_stack_retraining.py
& $mp2py stack_v2.py
```

Training additionally requires PyTorch 2.4.0 and scikit-learn 1.7.2. Existing runs/improve_v14/oof_0.npy through oof_4.npy are reused. To recompute them, use a clean copy and retain existing evidence separately. New combiners, selection records and validation scores are written to runs/improve_v14.

This is not end-to-end reconstruction of every final asset: the full-training PCA and five full-training base weights still come from the supplied assets. Base-training source is retained, but this delivery does not provide one command that recreates every historical selection from scratch. Fold and full-data training also differ in details such as batch size. Exact reproduction of 395 correct answers should load the frozen assets; retraining and reselection may differ with numerical libraries and need not yield bitwise or per-question identical results.

Files and Evidence

Material	Repository location
Inference and assets	code/submission_stack.py and code/model_bundle_stack/
Frozen source	code/model_bundle_stack_manifest.json
Official score and timing	code/results_stack/
Test evaluation history	code/improvement_history.json
Training and features	code/stack_v2.py; binary_v2.py; group_v2.py robust_v2.py; boost_v2.py; features_v2.py; features_v3.py

AI Assistance

AI assistance contributed to code improvements, revision suggestions, some methods not covered in the lecture slides, analysis of experiment records and the initial draft of this report.